

Lab #2

EGRE246

The following program defines type `intList` and two routines – `addItem` which adds items to the list, and `printList`. Your task during this lab is to write the single function `addItem` (within the program given); note that your list begins empty and then grows as needed at runtime (using `malloc`) whenever you add an item and discover there's no more room in the old list. The general strategy for doing this: if you're trying to add another item to the list and you find that the list is full, you create a new temporary list `CHUNK` (which is defined in the program) cells larger than the full one, then copy the items from the original list to this new bigger temporary one, then assign this new bigger list back to be the new list (i.e. you replace the old list with this new temporary one by just the assigning list pointers). The algorithm for doing this is given below.

Write the function `addItem` as defined below for the program given; note that the program in the lecture notes should give you a lot of help on how to do this! Your general algorithm for `addItem` (where '`←`' is assignment; see the code at the end for the definition of all variables and fields used in this algorithm):

```
if the intList l can't hold any more items then
    int *temp ← (new array created with malloc CHUNK cells bigger than l->data)
    copy all the old items from l->data over to temp
    store the pointer to the old array so you can free it up later: int *old ← l->data
    l->data ← temp
    l->listSize ← l->listSize + CHUNK
    free up old pointer
endif
assign item to the l->count cell in l->data
increment l->count
```

Here is the program you should use (no need to type it in – you can download it from Canvas).

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define CHUNK 10
#define DEBUG true

typedef struct {
    int *data;
    int listSize; // current number of cells in array
    int count;    // current number of items in list
} intList;

void addItem(intList*,int);
```

```

void printList(intList*);

int main(void){
    intList *l = malloc(sizeof(intList));
    l->data = NULL; l->count = l->listSize = 0;

    for (int i=1; i <= 112; i++)
        addItem(l,i*2);
    printList(l); printf("\n");
    return 0;
}

void addItem(intList *l, int item) {
    // your code goes here
}

void printList(intList *l) {
    for (int i=0; i<l->count; i++) {
        printf("%d ",l->data[i]);
    }
}

```